

TERCER LLIURAMENT PROP

SUBGRUP 41.2

Versió 2

Abel Fernández Leiva (abel.fernandez.leiva)

Judit Giner Velazquez (judit.giner)

Aura Han Ruiz Sánchez (aura.han.ruiz)

Paula Torreblanca Costa (paula.torreblanca)

INDEX

ESTRUCTURES DE DADES.....	2
IMPLEMENTACIÓ DE L'ALGORISME.....	3
HERÈNCIA/INTERFÍCIES.....	5
FUNCIONAMENT DE SILHOUETTE.....	6
DESCRIPCIÓ DE MÈTODES I ATRIBUTS DE LA CAPA DE DOMINI.....	7
DESCRIPCIÓ DE JOCS DE PROVES (DRIVERS).....	28
DESCRIPCIÓ DE JOCS DE PROVES (JUnit).....	31
DESCRIPCIÓ DE LES CLASSES DE LA CAPA DE PRESENTACIÓ.....	32
DESCRIPCIÓ DE LA CLASSE DE LA CAPA DE PERSISTÈNCIA.....	33
DESCRIPCIÓ DE CASOS D'ÚS.....	34
MANUAL D'USUARI.....	36
DIAGRAMA CASOS D'ÚS.....	37
DIAGRAMA DE CLASSES.....	38
CLASSES IMPLEMENTADES PER CADASCÚ.....	41

ESTRUCTURES DE DADES

1. `Object[][] data`.

Consisteix en un array d'objectes on es guarden totes les respostes d'una enquesta, després de processar-les (eliminar valors nulls i outliers), per tal d'enviar al algoritme KMeans, que s'encarrega de definir els clusters..

La tria d'aquesta estructura es basa en dos motius principalment. Per una banda, necessitàvem una estructura capaç de tractar qualsevol tipus de dada, des de Double fins a String. Definir les dades com a `Object` ens permet mantenir-les totes en una mateixa estructura, la qual cosa redueix la complexitat del codi. Per altra banda, hem triat un Array en comptes d'una altra estructura per tal de facilitar l'accés a dades. Poder accedir mitjançant índex permet poder iterar per tots els valors d'una manera sencilla, que era el nostre principal objectiu.

2. `int[] variableType`.

És un array que guarda, per a cada columna relevant (que correspon a una pregunta) en una enquesta, l'índex que identifica el tipus de pregunta de la que es tracta. La raó per la qual hem triat Integers en comptes d'un altre tipus de variable és per simplificar el programa, ja que utilitzar Strings seria més exhaustiu. Hem triat els índexs manualment de la següent manera:

0 → tipus numèric

1 → tipus qualitativa ordenada, resposta única (en aquest cas la resposta consisteix en un array de dos elements: l'índex de la opció triada, i el número d'opcions que té la pregunta, les quals necessitem pels càlculs de distàncies).

2 → tipus qualitativa no ordenada, resposta única

3 → tipus qualitativa no ordenada, resposta múltiple

4 → tipus text lliure

IMPLEMENTACIÓ DE L'ALGORISME

Per tal de realitzar un clustering el més precís possible, es van contemplar diverses alternatives. Al principi es va realitzar una implementació del algorisme K-Means, utilitzant un càlcul de distàncies Euclidià. No obstant, ràpidament ens vam adonar que aquest mètode no era el més adient. Entre altres coses, K-Means té dificultats per treballar amb dades no numèriques, i, a més, és sensible a outliers, la qual cosa pot generar molts problemes en un context on no coneixem les dades i per tant no sabem si trobarem outliers. Per aquesta raó, vam adoptar un model K-Means amb algunes variacions respecte l'original. Pel que fa al càlcul de distàncies, després de diverses proves, vam arribar a la conclusió de que fer la mitjana normalitzada de les dades ens proporcionava els millors resultats, tot i que aquests continuessin sense ser ideals.

El càlcul de la distància per a cada tipus de dada és el següent:

- Per a dades numèriques, inicialment calculaven la distància mitjançant una resta (i normalitzant el resultat d'aquesta utilitzant el màxim i mínim entre tots els valors d'aquella columna). El problema d'aquest enfocament és que les diferències eren pràcticament negligibles, per la qual cosa vam prendre la decisió d'eleva aquesta diferència al quadrat, fent que les distàncies prenguin valors més elevats-
- Les dades d'opció única ordenades es van tractar de forma similar. Com cada resposta té un index (ja que correspon a una posició concreta), el càlcul de la distància entre dues respostes és la diferència entre ambdós index, dividida entre el nombre total de respostes possibles per a normalitzar. D'aquesta manera, aconseguim resultats força realistes, tot i comptant amb la limitació que la similitud entre dues respostes veïnes no és sempre la mateixa.
- Per a dades de resposta única no ordenades vam prendre un enfocament molt directe. Si les dues respostes coincidien, la distància entre elles és 0 (són iguals). Si, en canvi, les respostes eren diferents, donavem un 1. Tot i que aquesta interpretació pot semblar lògica, ens preocupava que aquest tractament fós massa dràstic, però mitjançant diverses proves en les que donavem distàncies menors a 1 pels casos amb respostes diferents, ens vam adonar que els millors resultats els obteníem amb la nostra versió original.
- Pel que fa a les respostes d'opció múltiple vam decidir-nos per utilitzar la distància de Jaccard, que intenta averiguar com de semblar són dos grups de dades. Vam considerar diverses opcions, algunes més centrades en veure com de diferents

podien arribar a ser dos grups de dades, però després de veure que aquesta lògica baixava la precisió del nostre clustering, vam decidir optar per utilitzar Jaccard.

- Finalment, per a les respostes de text lliure vam començar amb una implementació de la distància Levenshtein. Aquesta es basa, principalment, en analitzar com de similars són dos strings des d'un punt de vista purament sintàctic. Aquest mètode, tal i com vam poder comprovar, no era massa eficient, ja que quan volem comprovar si dues respostes d'una enquesta són similars o no, ho pensem des del punt del significat. Per aquesta raó, de cara a la entrega final volíem millorar això. El nou enfocament que vam desenvolupar es basa en la distància del cosinus i enbeddings, que analitzen el significat de cada paraula i després mira si s'assembla a la resta. Aquest mètode ens va donar resultats infinitament més precisos per a datasets amb respostes d'aquest tipus. No obstant, el trade-off és que és un càlcul molt més pesat, i que per tant triga més. En els datasets que contenen respostes d'aquest tipus, el temps d'execució del clustering, que abans era < 1 segon, s'ha incrementat fins a 20 segons aproximadament. Tot i que aquest canvi és significatiu, hem considerat que preferim un anàlisi més exacta encara que es trgui més temps.

Per a la inicialització dels centroides (propia dels algorismes de clustering), vam utilitzar K-Means++. El principal problema que vam trobar amb aquest algorisme és que té un punt aleatori. Si bé és cert que és molt més efectiu que inicialitzar tots els centroides de forma aleatoria, segueix sent fortament aleatori, la qual cosa fa que no sigui completament fiable. K-Means++ parteix d'un centroide escollit a l'atzar, a partir del qual escull la resta basant-se en una aleatorietat probabilística. Una de les proves que vam fer és tractar d'eliminar aquesta aleatorietat en la tria de la resta de centroides, és a dir, utilitzar un enfocament més determinista. No obstant, aquesta mesura va tenir l'efecte contrari al esperat: els resultats del nostre clustering van ser pitjors. Amb això vam concloure que el algorisme que hem dissenyat es beneficia, fins a cert punt, d'aquesta aleatorietat esmentada.

Tenint ja un K-Means bastant eficient, trobavem encara algunes mancances. La realitat és que, tot i ser un algorisme molt potent, té algunes debilitats, tal i com s'ha mencionat al inici. Per aquesta raó, i buscant millorar l'eficiència del nostre algorisme, vam implementar un segon algorisme, K-Medoids, que suposa una millora respecte K-Means en certs aspectes.

La implementació de K-Medoids és força similar a K-Means, per la qual cosa no va ser complicat d'adaptar el nostre codi. La principal diferència és que mentres que K-Means utilitza la mitjana del cluster com a centroide, K-Medoids busca la dada més representativa

en cada cas. D'aquesta manera, l'algorisme és més robust a outliers i s'aconsegueixen així millors resultats en molts casos.

Després d'implementar aquest segon algorisme vam descobrir una cosa interessant; els clusterings en datasets estranys milloren amb K-Medoids, però aquells més ordinaris seguien millor amb K-Means. Així doncs, vam triar tots dos: s'executen un després de l'altre, i el que ens dona millor percentatge de precisió és el que ens quedem. Així aconseguim el millor clustering possible en cada cas.

En quant al tractament inicial de les dades, vam seguir un enfocament força senzill. Per una banda, els valors no computats els vam computar utilitzant o bé la moda o bé la mitjana, depenent del tipus de dada. Per altra banda, el tractament dels outliers va ser més complex. Inicialment vam utilitzar el valor del percentil 95 per re-computar els outliers, però vam trobar que aquest enfocament era subòptim, particularment en datasets petits (menys de 100 elements). Per aquesta raó vam buscar una alternativa, i finalment vam utilitzar l'IQR per computar valors que fossin molt distants a aquest. Aquest tractament va resultar molt millor per qualsevol tipus de dataset.

HERÈNCIA/INTERFÍCIES

En el relatiu a la part del algoritme dins del diagrama de classes, finalment vam escollir crear una classe algoritme abstracta amb dues subclasses (KMeans i KMedoids). La raó d'aquesta elecció és que ambdós algoritmes tenen parts de codi comunes, que per tant podem reaprofitar. Per altra banda, KMeans++ serà una classe que interaccionarà amb la classe algoritme, i ens servirà com a inicialitzador de qualsevol dels dos algoritmes. Totes tres classes esmentades (KMeans, KMedoids i KMeans++) utilitzaran una classe utilitària que serveix per calcular distàncies. Aquesta classe no tindrà instàncies, sinó que és una manera de reciclar codi de manera més neta.

FUNCIONAMENT DE SILHOUETTE

La mètrica de silhouette serveix per calcular la qualitat dels clusters generats pel K-means comparant la similitud entre els elements d'aquests. Per al seu càlcul s'utilitza la fórmula següent:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

On per un punt i:

- $b(i)$ és la distància mitjana mínima entre i i els elements d'un altre cluster
- $a(i)$ és la distància entre i i els elements del seu propi cluster
- $s(i)$ és l'indicador de qualitat del punt i

Una vegada aquesta fórmula s'ha aplicat per a tots els punts es treu la qualitat general del clustering calculant la mitjana de tots els punts de la següent manera:

$$S = \frac{1}{n} \sum_{i=1}^n s(i)$$

On:

- n és el nombre de punts totals
- S és la qualitat mitjana del clustering (resultat final silhouette)

Utilitzant aquesta fórmula hem pogut evaluar la qualitat de diverses instàncies de K-means i així decidir amb quina quedar-nos.

DESCRIPCIÓ DE MÈTODES I ATRIBUTS DE LA CAPA DE DOMINI

Algorithm.java

1. assignToCluster: mètode encarregat d'assignar cada resposta a un cluster.

Paràmetres d'entrada:

- Object[][] data → array que guarda les respostes que s'utilitzen per l'anàlisi.
- int[] variableType → array amb el tipus de pregunta de cada columna.

Valor de sortida:

- int[] → retorna, per cada resposta, l'index del cluster al que pertany.

2. initializeRandomCentroids: inicialitza, utilitzant l'algorisme de KMeans++, els centroides inicials per a executar KMeans.

Paràmetres d'entrada:

- Object[][] data → array que guarda les respostes que s'utilitzen per l'anàlisi.
- int n → nombre de respostes per a l'enquesta a analitzar.
- int[] variableType → array amb el tipus de pregunta de cada columna.
- double[] max → per a les columnes de tipus numèric, indica el màxim valor.
- double[] min → per a les columnes de tipus numèric, indica el mínim valor.

Valor de sortida:

- Object[][] → retorna els centroides inicials que ha indicat KMeans++ (com a files senceres).

3. findNearestCentroid: troba el centroide del cluster més proper a una fila indicada.

Paràmetres d'entrada:

- int i → indica l'index de la fila que es vol analitzar.
- int[] variableType → array amb el tipus de pregunta de cada columna.
- double[] max → per a les columnes de tipus numèric, indica el màxim valor.
- double[] min → per a les columnes de tipus numèric, indica el mínim valor.
- Object[][] data → array que guarda les respostes que s'utilitzen per l'anàlisi.

Valor de sortida:

- int → retorna l'index del cluster al que pertany la dada determinada.

4. calculateCentroids: recalcula els centroides de cada cluster després de cada iteració.

Paràmetres d'entrada:

- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.
- `int n` → nombre de respostes per a l'enquesta a analitzar.
- `int[] variableType` → array amb el tipus de pregunta de cada columna.

KMeans.java

1. `calculateCentroid`: retorna, per un cluster, un centroide que millor representa al cluster.

Paràmetres d'entrada:

- `List<Integer> clusterIndexes` → llista amb els índexs de totes les dades que pertanyen a un determinat cluster.
- `int[] variableType` → array amb el tipus de pregunta de cada columna.
- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.

Valor de sortida:

- `Object[][]` → retorna el centroide que millor representa un cluster.

KMedoids.java

2. `calculateMedoid`: retorna, per un cluster, el punt (existent) que millor representa un cluster.

Paràmetres d'entrada:

- `List<Integer> clusterIndexes` → llista amb els índexs de totes les dades que pertanyen a un determinat cluster.
- `int[] variableType` → array amb el tipus de pregunta de cada columna.
- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.

Valor de sortida:

- `Object[][]` → retorna el medoide d'un cluster.

KMeansPlusPlus.java

1. `initializeCentroids`: mètode encarregat d'inicialitzar els centroides dels clústers utilitzant l'algoritme KMeans++.

Paràmetres d'entrada:

- `Object[][] data` → array que guarda les respostes que s'utilitzen per l'anàlisi.
- `int k` → nombre de clusters que hi ha.
- `int[] variableType` → array amb el tipus de pregunta de cada columna.
- `double[] max` → per a les columnes de tipus numèric, indica el màxim valor.

- double[] min → per a les columnes de tipus numèric, indica el mínim valor.

Valor de sortida:

- int[] → retorna els centroides inicials trobats.

2. calculateMinDistances: calcula la distància mínima entre un punt i el seu centroide més proper.

Paràmetres d'entrada:

- Object[][] data → array que guarda les respostes que s'utilitzen per l'anàlisi.
- Object[][] centroids → array amb tots els centroides trobats fins el moment.
- int centroidsCount → nombre de centroides trobats fins al moment.
- int[] variableType → array amb el tipus de pregunta de cada columna.
- double[] max → per a les columnes de tipus numèric, indica el màxim valor.
- double[] min → per a les columnes de tipus numèric, indica el mínim valor.

Valor de sortida:

- double[] → retorna la distància per cada dada al seu centroide més proper.

3. chooseNextCentroid: calcula el següent centroide en basa a la probabilitat elevada al quadrat de les distàncies.

Paràmetres d'entrada:

- double[] distances → array amb les mínimes distàncies de cada dada a un centroide.
- Random random → element random que ens dona una probabilitat aleatòria.

Valor de sortida:

- int → retorna l'index del centroide trobat.

CalculateDistance.java

1. compute → retorna la distància normalitzada entre dues dades.

Paràmetres d'entrada:

- Object[] point → respostes d'una fila.
- Object[] centroid → centroide, respostes d'una altra fila.
- int[] variableType → array amb el tipus de pregunta de cada columna.
- double[] max → per a les columnes de tipus numèric, indica el màxim valor.
- double[] min → per a les columnes de tipus numèric, indica el mínim valor.

Valor de sortida:

- double → retorna la distància entre els dos punts.

2. numericDistance: calcula la diferència entre dos valors. Aquesta diferència és posteriorment normalitzada i s'eleva al quadrat per incrementar les diferències.

Paràmetres d'entrada:

- Object point → resposta numèrica.
- Object centroid → centroide, resposta numèrica de referència.
- double max → màxim de la columna a la que pertanyen les dades.
- double min → mínim de la columna a la que pertanyen les dades.

Valor de sortida:

- double → retorna la distància entre els dos punts.

3. singleChoiceOrdered: calcula la distància entre els índexs de dues respostes (ordenades) i la normalitza.

Paràmetres d'entrada:

- Object point → resposta única, ordenada.
- Object centroid → centroide, resposta única, ordenada, de referència.

Valor de sortida:

- double → retorna la distància entre els dos punts.

4. singleChoiceUnordered: calcula si dos respostes són iguals o no.

Paràmetres d'entrada:

- Object point → resposta única.
- Object centroid → centroide, resposta única de referència.

Valor de sortida:

- double → retorna 1 si la resposta no és la mateixa, 0 altrament.

5. multipleChoiceUnordered: calcula la semblança entre dos respostes multiresposta.

Paràmetres d'entrada:

- Object point → resposta múltiple.
- Object centroid → centroide, resposta múltiple de referència.

Valor de sortida:

- double → retorna la semblança normalitzada entre dos respostes.

6. freeText: calcula la semblança entre dos respostes de text lliure.

Paràmetres d'entrada:

- Object point → resposta de text lliure.
- Object centroid → centroide, resposta de text lliure de referència.

Valor de sortida:

- double → retorna la semblança normalitzada entre dos respostes.

KMeansDriver.java

1. main:

Paràmetres d'entrada:

- String[] args → qualsevol paràmetre que l'usuari pugui utilitzar en l'execució.

TestAccuracy.java

1. testAccuracyIris: calcula la precisió del clustering per a les dades de iris.csv

Paràmetres d'entrada:

- int[] clusters → array amb l'index del cluster al que pertany cada fila de les dades.
- List<Object> labels → llista amb la classificació real de cada fila de les dades.

2. testAccuracyLoanLoader: calcula la precisió del clustering per a les dades de loan-loader.csv

Paràmetres d'entrada:

- int[] clusters → array amb l'index del cluster al que pertany cada fila de les dades.
- List<Object> labels → llista amb la classificació real de cada fila de les dades.

AuxiliarMethods.java

1. levenshtein: calcula la distància de levenshtein entre dos strings.

Paràmetres d'entrada:

- String s1 → primer string.
- String s2 → segon string.

Valor de sortida:

- int → retorna la distància levenshtein entre dos strings.

2. calculateMode: calcula la moda per una llista d'elements de qualsevol tipus.

Paràmetres d'entrada:

- List<T> choices → llistat d'elements.

Valor de sortida:

- T → retorna l'element més freqüent en la llista.

3. calculateAverage: calcula la mitjana dels elements d'una llista.

Paràmetres d'entrada:

- List<Double> data → llistat d'elements.

Valor de sortida:

- Double → retorna la mitjana de tots els valors de la llista.

4. calculateMinVector: calcula, per a les columnes numèriques, el mínim valor de la columna.

Paràmetres d'entrada:

- Object[][] data → array que guarda les respostes que s'utilitzen per l'anàlisi.
- int[] variableType → array amb el tipus de pregunta de cada columna.

Valor de sortida:

- double[] → retorna un vector on les posicions que corresponen a columnes numèriques es guarda el mínim de la columna, altrament es guarda un -1.

5. calculateMaxVector: calcula, per a les columnes numèriques, el màxim valor de la columna.

Paràmetres d'entrada:

- Object[][] data → array que guarda les respostes que s'utilitzen per l'anàlisi.
- int[] variableType → array amb el tipus de pregunta de cada columna.

Valor de sortida:

- double[] → retorna un vector on les posicions que corresponen a columnes numèriques es guarda el màxim de la columna, altrament es guarda un -1.

6. calculateMax95P: calcula el valor del percentil 95 de la llista.

Paràmetres d'entrada:

- List <Double> list → llista dels valors.

Valor de sortida:

- Double → retorna el valor del nombre que correspon al percentil 95 de tots els de la llista.

Question.java -> Abstracta

1. Question: crea una instància de pregunta amb el seu enunciat i tipus corresponent. Aquests són els dos únics elements que tenen totes les classes concretes derivades d'aquesta, que és ABSTRACTE.

Paràmetres d'entrada:

- String enunciat: Enunciat de la pregunta.
- Int tipus: aquest enter ens indica quin tipus de pregunta és:
 - 1 = Resposta única (SingleChoiceQuestion)
 - 2 = Resposta múltiple (MultipleChoiceQuestion)
 - 3 = Text lliure (FreeTextQuestion)
 - 4 = Número lliure (FreeNumberQuestion)

2. crearQuestion: una vegada hem creat la instància de pregunta, hem de dir de quin tipu és. La classe abstracta no és una classe com a tal per ella sola, és **complete disjoint**. Aquesta classe fa que en el nostre codi podem crear una Question q i igualar-la a aquest mètode, que retorna una instància de la subclasse triada.

Aquesta classe té 3 funcions abstractes que es sobreescrueixen/ es fa override en cada subclasse: validarResposta, toString i ModifyQuestion. L'única que no està buida és toString que sempre retorna l'enunciat i el tipus de pregunta (valors de la classe pare i que, per tant, tenen les classes filles). Aquesta funció retorna una String. És bàsicament per interacció de qualitat amb l'usuari si executa el programa desde terminal i per debugació.

SingleChoiceQuestion

1. SingleChoiceQuestion: constructora de pregunta d'única resposta.

Paràmetres d'entrada:

- String enunciat: Enunciat de la pregunta.
- List<String> opcions: llista de respostes que té la pregunta

Sortida:

- S'ha creat una pregunta amb enunciat i diverses opcions de resposta. A més, es posa tipus = 1.

2. `validarResposta`: comprova que la resposta sigui vàlida. Quan responem una pregunta mirem si la resposta és realment un índex enter igual o més gran a zero i que es doni un índex menor al total d'opcions.

Paràmetres d'entrada:

- Object `resposta`: la resposta que rebem. Verifiquem que sigui de tipus `int`.

Paràmetres de sortida:

- Bool → True si la resposta a la pregunta és vàlida. Fals si no ho és.
3. `toString`: En el cas de pregunta de resposta única retorna les opcions que es poden respondre, a més del propi enunciat i el tipus, que agafa de `toString` de la classe pare.
 4. `ModifyQuestion`: Imprimeix les opcions actuals i deixa posar-ne de noves, tal que es borren totes les anteriors i queden com a opcions les noves introduïdes separades per “;”.

MultipleChoiceQuestion

Aquest tipus de pregunta consisteix en una pregunta amb diverses respostes, de les quals es pot triar un cert nombre de respostes, o totes. Per això té un `int maxRespostes` que indica el màxim de respostes que podem triar. Si aquest valor és -1 vol dir que podem triar-les totes.

1. `MultipleChoiceQuestion`: igual que la constructora de la `SingleChoiceQuestion`, però amb un enter que indica el màxim de respostes que podem triar.

Paràmetres d'entrada: String enunciat, List<String> opcions, int maxRespostes.

2. `validarResposta`:

Paràmetres d'entrada: iguals que per `SingleChoiceQuestion`. Object `resposta`.

Paràmetres de sortida:

Bool → True si el paràmetre d'entrada és un vector d'enters. False si la llargada del vector d'índexs de respostes triades és més llarg que `maxRespostes`.

3. `toString`: igual que `SingleChoiceQuestion`, però la string retornada al principi indica el màxim de respostes que es pot triar.
4. `ModifyQuestion`: igual que `SingleChoiceQuestion`, però afegint que també es pot canviar el número màxim de respostes.

FreeTextQuestion

Aquest tipus de classe no té altres atributs que no tingui la pròpia classe Question. Es tracta d'un enunciat i un tipus de pregunta.

1. FreeTextQuestion: agafa la constructora de la superclasse i hi posa el número 3 que correspon a aquest tipu de pregunta.
2. validarResposta:
Paràmetres d'entrada: igual en totes, Object resposta.
Paràmetre de sortida: Bool -> True si tenim una cadena de caràcters de mida igual o menor a 100. També s'hi val posar un número com a resposta en aquesta subclasse.
3. ModifyQuestion: Només canvia l'enunciat ja que no conté respostes a triar.

FreeNumberQuestion

Molt semblant a la FreeTextQuestion.

1. FreeNumberQuestion: agafa la constructora de la superclasse i hi posa el número 4 que correspon a aquest tipu de pregunta.
2. validarResposta:
Paràmetres d'entrada: igual en totes, Object resposta.
Paràmetre de sortida: Bool -> True si entra com a resposta un enter o decimal.
3. ModifyQuestion: Només canvia l'enunciat ja que no conté respostes a triar.

Survey.java

1. Survey: Crea una instància de la classe Survey sense inicialitzar
Paràmetres d'entrada: cap
Valor de sortida:
 - La nova instància no inicialitzada de Survey
2. Survey: Crea una instància inicialitzada de Survey
Paràmetres d'entrada:
 - Id: identificador de l'enquesta
 - Descripcio: descripció de l'enquesta
 - Títol: títol de l'enquestaValor de sortida: la nova instància de survey inicialitzada
3. CanviTítol: Modifica el títol de l'enquesta del antic a un nou, insertat per terminal
Paràmetres d'entrada: cap
Valor de sortida: cap(void)

4. CanviDescr: Modifica la descripció de l'enquesta de la antiga a una nova, instertada per terminal.
Paràmetres d'entrada: cap
Valor de sortida: cap(void)
5. gestionarPreg: Permet modificar, esborrar o ignorar cada una de les preguntes de l'enquesta.
Paràmetres d'entrada: cap
Valor de sortida: cap(void)
6. afegirPreg: Permet decidir si volem afegir una nova pregunta i crear-la
Paràmetres d'entrada: cap
Valor de sortida: cap(void)
7. IntToObject: Converteix un array de int a un array de Object.
Paràmetre d'entrada:
 - Array de intValor de sortida: Array de Object
8. ModifySurvey: Permet decidir si modificar el títol o la descripció, afegir noves preguntes i, si hi han preguntes ja existents modificar-les o esborrar-les.
Paràmetres d'entrada: cap
Valor de sortida: cap
9. SurveyAnalysis: Analitza l'enquesta i agrupa les seves respostes en clusters utilitzant kmeans
Paràmetres d'entrada: cap
Valor de sortida: cap
10. SurveyAnswer: Crea una nova instància de Answer i la inicialitza amb les respostes introduïdes per terminal.
Paràmetres d'entrada:cap
Valor de sortida: cap

Analysis.java

1. Analysis: Creadora sense inicialitzar de la classe.
Paràmetres d'entrada: cap
Valor de sortida: nova instància de la classe Analysis.
2. Initialize: inicialitza els clusters de la classe
Paràmetres d'entrada:

- `Int[]` clusters: valors dels clusters

Valor de sortida: cap

3. Silhouette: Examina la qualitat dels clusters de diferents instàncies de kmeans i selecciona el millor.

Paràmetres d'entrada:

- `Int[]` result: resultat del kmeans
- `Object[][]` punts: posició dels punts sobre els quals es calcula kmeans
- `Int[]` type: array amb els tipus de pregunta
- `Double[]` min: array amb els valor mínims
- `Double[]` max: array amb els valors màxims
- `Int` k: k trada pel k-means

Answer.java

Atributs:

- `Object[]` answer - Array que emmagatzema les respostes d'un usuari a totes les preguntes
- `BufferedReader` reader - Objecte per llegir l'entrada de l'usuari des de la consola

Mètodes:

1. Constructors:

- `Answer()` - Constructor buit que inicialitza un array de respostes buit i el `BufferedReader`

2. inicialització i entrada de dades:

- `initialize(int[] type, ArrayList<Question> questions)` - Inicialitza les respostes demanant a l'usuari que respongui cada pregunta. Mostra cada pregunta i gestiona l'entrada segons el tipus.

Paràmetres d'entrada:

- `int[]` type: llista de tipus
- `ArrayList<Question>` questions: array de preguntes

Valor de sortida: cap

3. Mètodes de lectura d'entrada:

- `readSingleChoiceOrdered(Question question)` - Llegeix una resposta d'opció única ordenada (per índex)

Paràmetres d'entrada:

- Question question: objecte pregunta

Valor de sortida:

- Object[] result de mida 2, on result[0] és l'índex seleccionat i result[1] el total d'opcions, si la resposta és vàlida
- Object[0], si la resposta no és vàlida
- readSingleChoiceUnordered(Question question) - Llegeix una resposta d'opció única no ordenada (per text)

Paràmetres d'entrada:

- Question question: objecte pregunta

Valor de sortida:

- String: text de resposta, si la resposta és vàlida
- "", si la resposta no és vàlida
- readMultipleChoice(Question question) - Llegeix respostes múltiples (per índexs separats per coma)

Paràmetres d'entrada:

- Question question: objecte pregunta

Valor de sortida:

- Set<Object> de mida variable, on cada Objecte és una resposta seleccionada per índex, si la resposta és vàlida
- HashSet<>(), si la resposta no és vàlida
- readFreeText() - Llegeix una resposta de text lliure

Paràmetres d'entrada:

- Question question: objecte pregunta

Valor de sortida:

- String: text de resposta
- readFreeNumber() - Llegeix una resposta numèrica

Paràmetres d'entrada:

- Question question: objecte pregunta

Valor de sortida:

- Object: num com a resposta, si la resposta és vàlida
- null, si la resposta no és vàlida

4. getters i setters:

- setAnswer(Object[] answer) - Estableix l'array de respostes donat com el propi de la classe

Paràmetres d'entrada:

- Object[] answer: llista de respostes a una enquesta

Valor de sortida: cap

- getArray() - Retorna l'array de respostes

Funcionalitat principal:

La classe Answer s'encarrega de:

- Capturar les respostes d'un usuari a través de la interfície
- Validar que les respostes coincideixin amb el tipus de pregunta esperat
- Gestionar diferents formats d'entrada segons el tipus de pregunta
- Emmagatzemar les respostes en un array d'Object per emmagatzemar múltiples tipus de dades

Tipus de respostes suportats:

- Tipus 0 (FreeNumber): Números (Double)
- Tipus 1 (SingleChoiceOrdered): llista de dos objectes, result[0]=índex, result[1]=numopcions
- Tipus 2 (SingleChoiceUnordered): Text amb el valor exacte de l'opció
- Tipus 3 (MultipleChoice): Set d'objectes amb els índexs seleccionats
- Tipus 4 (FreeText): Text lliure (String)

CONTROLADOR

CtrlDomini.java

Controlador de domini que gestiona enquestes, preguntes i respostes.

Centralitza tota la lògica de domini.

Atributs:

- List<Survey> surveyList

Llista que emmagatzema totes les enquestes disponibles del sistema

- GestorDeDades gestor

Instància del gestor de persistència per guardar/carregar dades JSON

- Survey currentSurvey

Referència a l'enquesta que s'està creant o modificant actualment

Mètodes:

Constructors:

- CtrlDomini()

Constructor buit que inicialitza la llista d'enquestes sense carregar dades del disc

Paràmetres d'entrada: cap

Valor de sortida: cap

- CtrlDomini(boolean loadFromDisk)

Constructor que opcionalment carrega enquestes persistides

Paràmetres d'entrada:

- boolean loadFromDisk - indica si es carreguen les enquestes del disc

Valor de sortida: cap

Inicialització:

- initializeCtrlDomini(Survey[] surveys)

Inicialitza el controlador amb un array d'enquestes existents

Paràmetres d'entrada:

- Survey[] surveys - array d'enquestes a carregar

Valor de sortida: cap

Gestió d'enquestes:

- SurveyCreation(String title, String desc)

Crea una nova enquesta i l'estableix com enquesta actual

Paràmetres d'entrada:

- String title - títol de l'enquesta
- String desc - descripció de l'enquesta

Valor de sortida: cap

- modificarEnquesta(String nouTitol, String novaDesc)

Modifica el títol i descripció de l'enquesta actual

Paràmetres d'entrada:

- String nouTitol - nou títol de l'enquesta
- String novaDesc - nova descripció de l'enquesta

Valor de sortida: cap

- eliminarEnquesta(String title)

Elimina una enquesta del sistema per títol

Paràmetres d'entrada:

- String title - títol de l'enquesta a eliminar

Valor de sortida: cap

- setCurrentSurveyByTitle(String title)

Estableix com actual l'enquesta amb el títol especificat

Paràmetres d'entrada:

- String title - títol de l'enquesta a seleccionar

Valor de sortida: cap

- setCurrentSurveyById(String idStr)

Estableix com actual l'enquesta amb l'ID especificat

Paràmetres d'entrada:

- String idStr - ID de l'enquesta a seleccionar

Valor de sortida: cap

- importSurvey(String filePath, String title, String description)

Importa una enquesta des d'un fitxer CSV

Paràmetres d'entrada:

- String filePath - camí del fitxer CSV
- String title - títol per a l'enquesta importada

- String description - descripció per a l'enquesta importada

Valor de sortida: cap

Gestió de preguntes:

- afegirPreguntaSimple(String enunciado, String tipus)

Afegeix una pregunta simple (text o numèrica sense opcions) a l'enquesta actual

Paràmetres d'entrada:

- String enunciado - text de la pregunta
- String tipus - tipus de pregunta ("Text Lliure" o "Numèrica")

Valor de sortida: cap

- afegirPreguntaAmbOpciones(String enunciado, String tipus, List<String> opciones, int maxRespuestas)

Afegeix una pregunta amb opcions a l'enquesta actual

Paràmetres d'entrada:

- String enunciado - text de la pregunta
- String tipus - tipus de pregunta
("Opció Única Ordenada", "Opció Única Desordenada", "Opció Múltiple")
- List<String> opciones - llista d'opcions disponibles
- int maxRespuestas - màxim de respostes per a preguntes múltiples (-1 = sense límit)

Valor de sortida: cap

- modificarPregunta(int index, String enunciado, String tipus, List<String> opciones, int maxRespuestas)

Modifica una pregunta existent, permetent canviar-ne el tipus

Paràmetres d'entrada:

- int index - índex de la pregunta a modificar
- String enunciado - nou text de la pregunta
- String tipus - nou tipus de pregunta
- List<String> opciones - nova llista d'opcions
- int maxRespuestas - nou límit de respostes (si aplicable)

Valor de sortida: cap

- eliminarPregunta(int index)

Elimina una pregunta de l'enquesta actual

Paràmetres d'entrada:

- int index - índex de la pregunta a eliminar

Valor de sortida: cap

- afegirPreguntaBuidaAlFinal(String tipus)

Afegeix una pregunta buida al final de l'enquesta actual

Paràmetres d'entrada:

- String tipus - tipus de pregunta a crear

Valor de sortida: cap

- getNumPreguntesActual()

Retorna el nombre de preguntes de l'enquesta actual

Paràmetres d'entrada: cap

Valor de sortida: int - nombre de preguntes

- getPreguntaData(int index)

Retorna les dades serialitzades d'una pregunta específica

Paràmetres d'entrada:

- int index - índex de la pregunta

Valor de sortida: String - dades de la pregunta serialitzades

Gestió de respostes:

- afegirResposta(Answer a)

Afegeix una resposta a l'enquesta actual

Paràmetres d'entrada:

- Answer a - objecte resposta

Valor de sortida: cap

- tractarEnviarResposta(Object[] arr)

Afegeix una resposta des d'un array d'objectes

Paràmetres d'entrada:

- Object[] arr - array amb les respostes

Valor de sortida: cap

Anàlisi i clustering:

- tractarAnalisi(String kStr)

Executa l'anàlisi de clustering K-means sobre l'enquesta actual

Paràmetres d'entrada:

- String kStr - valor de k (nombre de clusters)

Valor de sortida: String - resultat de l'anàlisi en format text

- tractarFerAnalisiRespostes(String filePath)

Executa clustering sobre respostes importades d'un CSV

Paràmetres d'entrada:

- String filePath - camí del fitxer CSV amb les respostes

Valor de sortida: String - resultat de l'anàlisi en format text

- analyzeAnswers(String filePath)

Analitza estadístiques de les respostes d'un fitxer CSV

Paràmetres d'entrada:

- String filePath - camí del fitxer CSV

Valor de sortida: String - estadístiques de l'anàlisi

Getters:

- getSurveyList()

Retorna la llista de totes les enquestes

Paràmetres d'entrada: cap

Valor de sortida: List<Survey> - llista d'enquestes

- getCurrentSurvey()

Retorna l'enquesta actualment seleccionada

Paràmetres d'entrada: cap

Valor de sortida: Survey - enquesta actual (pot ser null)

- getCurrentSurveyTitle()

Retorna el títol de l'enquesta actual

Paràmetres d'entrada: cap

Valor de sortida: String - títol de l'enquesta actual

- getCurrentSurveyDescription()

Retorna la descripció de l'enquesta actual

Paràmetres d'entrada: cap

Valor de sortida: String - descripció de l'enquesta actual

- getSurveyTitles()

Retorna una llista amb els títols de totes les enquestes

Paràmetres d'entrada: cap

Valor de sortida: List<String> - títols de les enquestes

- getSurveysFullInfo()

Retorna informació completa de totes les enquestes (ID, títol, descripció)

Paràmetres d'entrada: cap

Valor de sortida: List<String> - informació completa de les enquestes

- getCurrentSurveyQuestionsData()

Retorna les dades de totes les preguntes de l'enquesta actual

Paràmetres d'entrada: cap

Valor de sortida: List<String> - dades de les preguntes

Mètodes auxiliars (privats):

- crearInstanciaPregunta(String enunciado, String tipus, List<String> opcions,
int maxRespostes)

Crea una instància de pregunta segons el tipus especificat

Paràmetres d'entrada:

- String enunciado - text de la pregunta
- String tipus - tipus de pregunta

- List<String> opcions - llista d'opcions
- int maxRespostes - límit de respostes (si aplicable)

Valor de sortida: Question - objecte pregunta creat

- saveSurveys()

Guarda totes les enquestes a persistència (JSON)

Paràmetres d'entrada: cap

Valor de sortida: cap

readsurvey.java

Classe per llegir i importar enquestes des de fitxers CSV.

Processa format estàndard de preguntes.

Atributs:

Cap atribut de classe (classe amb mètodes estàtics)

Mètodes públics estàtics:

- createSurveyFromCSV(String csvFilePath)

Llegeix un fitxer CSV i crea una llista de preguntes

Paràmetres d'entrada:

- String csvFilePath - camí del fitxer CSV a llegir

Valor de sortida: ArrayList<Question> - llista de preguntes creades

Format CSV esperat:

type,statement,options,maximum

- type: 0-4 (tipus de pregunta)
- statement: text de la pregunta
- options: opcions separades per | (per a tipus 1, 2, 3)
- maximum: límit de respostes (només per a tipus 3)

Mètodes privats estàtics:

- parseCSVLine(String line)

Parseja una línia CSV, manejant camps entrecomats

Paràmetres d'entrada:

- String line - línia CSV a processar

Valor de sortida: List<String> - camps parseados de la línia

- createQuestionFromFields(int type, String statement, String optionsStr, String maxStr)

Crea un objecte Question a partir dels camps parseados

Paràmetres d'entrada:

- int type - tipus de pregunta (0-4)
- String statement - text de la pregunta
- String optionsStr - string amb opcions separades per |
- String maxStr - límit de respostes en format text

Valor de sortida: Question - objecte pregunta creat (pot ser null si hi ha error)

TIPUS DE PREGUNTES SUPORTADES:

- Tipus 0: Pregunta numèrica lliure (FreeNumberQuestion)
- Tipus 1: Opció única ordenada (SingleChoiceQuestionOrdered)
- Tipus 2: Opció única desordenada (SingleChoiceQuestionUnordered)
- Tipus 3: Opció múltiple (MultipleChoiceQuestion)
- Tipus 4: Text lliure (FreeTextQuestion)

EXEMPLE DE FORMAT CSV:

0,Quantes hores estudies per setmana?,
1,Matèria preferida?,Matemàtiques|Física|Literatura
2,Sistema operatiu?,Windows|macOS|Linux
3,Fruïtes que t'agraden?,Poma|Plàtan|Taronja|Raïm,2
4,Describeix el teu ambient ideal d'estudi,

DESCRIPCIÓ DE JOCS DE PROVES (DRIVERS)

KMeansDriver.java

Driver per executar proves del sistema de classificació mitjançant KMeans.

Objecte de la prova:

Aquest executable prova la integració final del procés de clustering mitjançant l'algorisme KMeans. Primer de tot extreu les dades d'un fitxer (a elecció de l'usuari) i el tipus de pregunta al que correspon cada dada. A continuació, es processen les dades d'aquest fitxer de manera que s'eliminen valors nuls i es modifiquen els outliers. Finalment, s'envien les dades a KMeans per a realitzar el clustering.

- Seleccionar l'enquesta per respondre (escollint l'arxiu a analitzar).
- Seleccionar la k (nombre de clusters en els que es divideixen les dades).
- Veure l'anàlisi (obrint el fitxer results.csv, on es guarda el clustering).

Altres elements integrats a la prova:

- ReadAnswers.java per llegir les dades del fitxer i fer el pre-processament.
- KMeans.java per a executar l'algoritme que fa el clustering.
- AuxiliarMethods.java per a utilitzar mètodes específics com ara càlcul de la moda o la mitjana.

Fitxers de dades necessaris:

- iris.csv: conté vectors de característiques numèriques, amb cada fila etiquetada amb el seu grup real entre iris-setosa, iris-versicolor, iris-virginica.
- loan-loader.csv: conté informació numèrica referent a dades de préstecs. Es fa servir per validar el funcionament de l'algorisme amb dades numèriques i de resposta única. A més, conté cada fila etiquetada amb el valor del cluster real.

Cada fila d'aquests fitxers representa un objecte que es converteix en un vector Objecte que després s'utilitza per l'algorisme.

Valors estudiats:

La prova utilitza les dades completes dels fitxers CSV:

- Caixa negra: es comprova la precisió del algorisme per retornar els resultats corresponents als reals (que coneixem gràcies a les etiquetes de cada fila) sense necessitat d'analitzar l'estructura interna del codi.

- Caixa blanca: es validen comportaments per a conjunts homogenis per comprovar que vectors similars acaben al mateix cluster.

L'objectiu és observar el comportament de KMeans en diferents dades i estudiar la coherència entre resultats esperats i resultats generats.

Efectes estudiats:

A nivell funcional, el driver gestiona:

- Interacció per consola per seleccionar el fitxer de dades.
- Lectura i preprocessament de les dades del fitxer.
- Execució del algorisme i obtenció de resultats (inclohent la precisió d'aquests):
- Generació d'un fitxer results.csv amb la classificació final.

Operativa:

1. El programa mostra un llistat amb els fitxers sobre els quals es pot fer l'anàlisi.
2. L'usuari selecciona el fitxer que vol analitzar.
3. L'usuari selecciona el valor de la k que vol.
4. El programa llegeix el CSV i calcula els clusters amb KMeans.
5. Es mostra per consola la precisió de l'algorisme.
6. Es guarden a results.csv els resultats del clustering.

CtrlDomini.java

Aplicació principal per gestionar, respondre i analitzar enquestes mitjançant una interfície d'interacció basada en consola.

Objecte de la prova:

Aquest executable prova la integració completa del sistema de gestió d'enquestes, incloent:

- Tots els casos d'ús relacionats amb la gestió d'enquestes (crear, modificar...).
- Resposta d'enquestes (de diferents tipus).
- Anàlisi d'enquesta (triada per l'usuari).

Altres elements integrats a la prova:

L'aplicació reutilitza diverses funcionalitats que ja han estat provades de forma individual.

- Survey.java, que s'encarrega de gestionar preguntes, respostes i anàlisis.
- Question.java i les seves subclasses per a diferents tipus de preguntes.

- Answer.java per capturar les respostes d'usuari.
- KMeansDriver.java per executar l'algorisme amb enquestes ja existents (el seu funcionament s'ha especificat a dalt).

Aquest executable combina aquestes classes per a comprovar el funcionament conjunt.

Fitxers de dades necessaris:

L'aplicació bàsica no requereix fitxers externs, però sí que depèn de les dades introduïdes per cada usuari per consola. Així doncs, l'executable no necessita carregar dades externes.

Valors estudiats:

L'executable permet provar el sistema amb qualsevol escenari que l'usuari desitgi. No obstant, cal garantir que almenys una dada introduïda és diferent de null per tal de realitzar l'anàlisi. S'aplica principalment una perspectiva de caixa negra per provar la interacció (ja que l'usuari que executa el codi no té perquè conèixer com s'ha implementat aquests. No obstant, trobem alguns casos de caixa blanca (comprovació d'una selecció de respostes correctes o recorregut de les dades).

Efectes estudiats:

A nivell funcional, el driver gestiona:

- Navegació per menús textuais i lectura d'opcions via consola.
- Interaccions de selecció per part de l'usuari (escollir enquesta, opció...).
- Fluxos complets per gestionar, respondre i analitzar una enquesta.
- Integració amb el sistema d'algoritmes d'anàlisi.

Operativa:

1. L'aplicació mostra un menú principal, donant a l'usuari l'opció de triar entre gestionar, respondre, analitzar enquesta o sortir del programa.
2. En funció de la opció triada, l'usuari pot:
 - Crear una enquesta.
 - Modificar o eliminar una enquesta existent (seleccionant el seu index).
 - Afegir preguntes a l'enquesta.
 - Fer un anàlisi KMeans de l'enquesta (triant l'enquesta i la k).

DESCRIPCIÓ DE JOCS DE PROVES (JUnit)

Tots els jocs de proves estan comentats a les seves respectives classes Test.java.
Es poden trobar a: `subgrup-prop41.2/FONTS/src/test/java`

DESCRIPCIÓ DE LES CLASSES DE LA CAPA DE PRESENTACIÓ

ControladorPresentacio.java

Classe responsable de gestionar la interacció entre l'usuari i el sistema. Rep les accions generades per les vistes i decideix el flux de navegació de l'aplicació, indicant en cada moment quina vista s'ha de mostrar. A més, realitza transformacions bàsiques de les dades provinents de la interfície (formats, conversions i validacions) abans de delegar la lògica de negoci al controlador de domini.

VistaPrincipal.java

Classe que actua com a contenidor principal de la interfície gràfica. Gestiona el canvi entre les diferents vistes mitjançant un CardLayout, però no decideix quina vista s'ha de mostrar en cada moment; aquesta decisió correspon exclusivament al controlador de presentació. La seva funció és executar els canvis visuals i mantenir el registre de vistes disponibles.

VistaMenu.java

Vista inicial que interactua directament amb l'usuari. Mostra les opcions principals del sistema i captura les accions de l'usuari mitjançant els botons de **Gestionar**, **Respondre** i **Anàlisi (d'enquestes o de respostes importades)**. Aquestes accions són sempre delegades al controlador de presentació, mantenint una clara separació entre la gestió de la interfície i la lògica de navegació.

VistaGestionar.java

Vista similar al menú principal que permet gestionar les enquestes existents. Conté opcions per **crear** una nova enquesta, **modificar-ne** una d'existent o **esborrar-la**. Totes les accions de l'usuari es deleguen al controlador de presentació, que decideix el flux posterior de l'aplicació.

VistaCrearEnquesta.java

Vista encarregada de la creació d'una nova enquesta. Conté camps de text per introduir el títol (obligatori) i la descripció (opcional). Un cop creada l'enquesta, el sistema navega cap a la vista d'edició de preguntes corresponent.

VistaEditorPreguntes.java

Vista destinada a l'edició d'una pregunta concreta d'una enquesta. Permet modificar l'enunciat i el tipus de pregunta. En el cas de preguntes amb opcions, permet afegir o eliminar opcions (amb un mínim d'una) i permet afegir un màxim de respostes a triar per al tipus múltiple. Per a preguntes de text o numèriques, només es permet modificar l'enunciat. També incorpora controls de navegació per desplaçar-se entre les diferents preguntes de l'enquesta i opcions per crear noves preguntes. Aquesta vista és dinàmica, ja que necessita informació específica de cada pregunta per construir la interfície.

VistaSeleccionarEnquesta.java

Vista dinàmica que mostra una llista de les enquestes existents i permet seleccionar-ne una, segons el context (modificar, respondre o analitzar). El comportament concret després de la selecció es delega al controlador de presentació.

VistaEsborrarEnquesta.java

Vista destinada a la confirmació de l'esborrat d'una enquesta. Mostra una llista d'enquestes i sol·licita confirmació explícita a l'usuari abans d'executar una acció destructiva.

VistaRespondreEnquesta.java

Vista dinàmica que permet respondre una enquesta pregunta per pregunta. Mostra l'enunciat i les opcions corresponents segons el tipus de pregunta, i disposa de controls de navegació per avançar o retrocedir entre preguntes. Les respostes es recullen i s'envien al controlador de presentació un cop finalitzat el procés.

VistaAnàlisis.java

Vista encarregada d'executar l'anàlisi de l'enquesta. Permet seleccionar el valor del paràmetre k i mostra el resultat de l'anàlisi mitjançant l'algorisme de *k-means*, delegant l'execució al controlador de presentació.

DESCRIPCIÓ DE LA CLASSE DE LA CAPA DE PERSISTÈNCIA

GestorDeDades.java

Gestor encarregat d'assegurar la persistència de les totes les dades del programa, tant de les preguntes com de les respostes. També emmagatzema les relacions que es produeixen entre aquestes doc classes i enquesta. Aquestes indiquen quines preguntes pertanyen a cada enquesta i quines respostes pertanyen a cada enquesta.

DESCRIPCIÓ DE CASOS D'ÚS

Nom: Gestionar enquesta

- Actor: Usuari
- Comportament: S'obre una pantalla on permet a l'usuari triar entre crear una nova enquesta o modificar-ne una d'existent.
- Errors possibles i cursos alternatius: -

Nom: Respondre enquesta

- Actor: Usuari
- Comportament: S'obre una pantalla amb totes les enquestes existents.
- Errors possibles i cursos alternatius: si no hi ha enquestes existents salta un missatge d'error.

Nom: Fer anàlisi

- Actor: Usuari
- Comportament: S'obre una pantalla amb totes les enquestes existents.
- Errors possibles i cursos alternatius: si no hi ha enquestes existents salta un missatge d'error.

Nom: Crear enquesta

- Actor: Usuari
- Comportament: S'obre una nova enquesta amb una sola pregunta en blanc
- Errors possibles i cursos alternatius: -

Nom: Modificar enquesta

- Actor: Usuari
- Comportament: S'obre una pantalla amb totes les enquestes existents.
- Errors possibles i cursos alternatius: si no hi ha enquestes existents salta un missatge d'error.

Nom: Seleccionar enquesta editar

- Actor: Usuari
- Comportament: S'obre l'enquesta seleccionada per la primera pregunta i permet a l'usuari editar-la.
- Errors possibles i cursos alternatius: -

Nom: Crear pregunta.

- Actor: Usuari
- Comportament: Afegeix una nova pregunta en blanc a l'enquesta seleccionada
- Errors possibles i cursos alternatius: -

Nom: Eliminar pregunta

- Actor: Usuari

- Comportament: Elimina la pregunta seleccionada
- Errors possibles i cursos alternatius: -

Nom: Seleccionar enquesta respondre

- Actor: Usuari
- Comportament: S'obre l'enquesta seleccionada i permet a l'usuari respondre les seves preguntes.
- Errors possibles i cursos alternatius: -

Nom: Seleccionar enquesta analitzar

- Actor: Usuari
- Comportament: s'obre una pàgina que et permet triar la k i veure l'anàlisi.
- Errors possibles i cursos alternatius: L'enquesta seleccionada ha de tenir alguna resposta, sinó en té salta un missatge d'error.

Nom: Triar k

- Actor: Usuari
- Comportament: Es tria la k per a fer l'anàlisi.
- Errors possibles i cursos alternatius: Si la k triada és major al nombre de respostes salta un missatge d'error.

Nom: Veure anàlisi

- Actor: Usuari
- Comportament: Es mostra l'anàlisi de l'enquesta triada utilitzant l'algoritme k-means amb la k indicada.
- Errors possibles i cursos alternatius: Si la k no ha estat triada, es calcula la millor opció utilitzant elbow method.

Nom: Editar següent pregunta

- Actor: Usuari
- Comportament: S'obre la pregunta següent i permet a l'usuari editar-la.
- Errors possibles i cursos alternatius: Si no existeix cap pregunta no fa res.

Nom: Editar pregunta anterior

- Actor: Usuari
- Comportament: S'obre la pregunta anterior i permet a l'usuari editar-la.
- Errors possibles i cursos alternatius: Si no existeix una pregunta anterior no fa res.

Nom: Sortir

- Actor: Usuari
- Comportament: Es desa l'enquesta editada i es torna a la pàgina inicial.
- Errors possibles i cursos alternatius: Si hi ha preguntes en blanc o no acabades, aquestes es desen en blanc.

Nom: Contestar següent pregunta

- Actor: Usuari
- Comportament: S'obre la pregunta següent i permet a l'usuari contestar-la.
- Errors possibles i cursos alternatius: Si no existeix una pregunta següent no fa res.

Nom: Contestar pregunta anterior

- Actor: Usuari
- Comportament: S'obre la pregunta anterior i permet contestar-la.
- Errors possibles i cursos alternatius: Si no existeix una pregunta anterior no fa res.

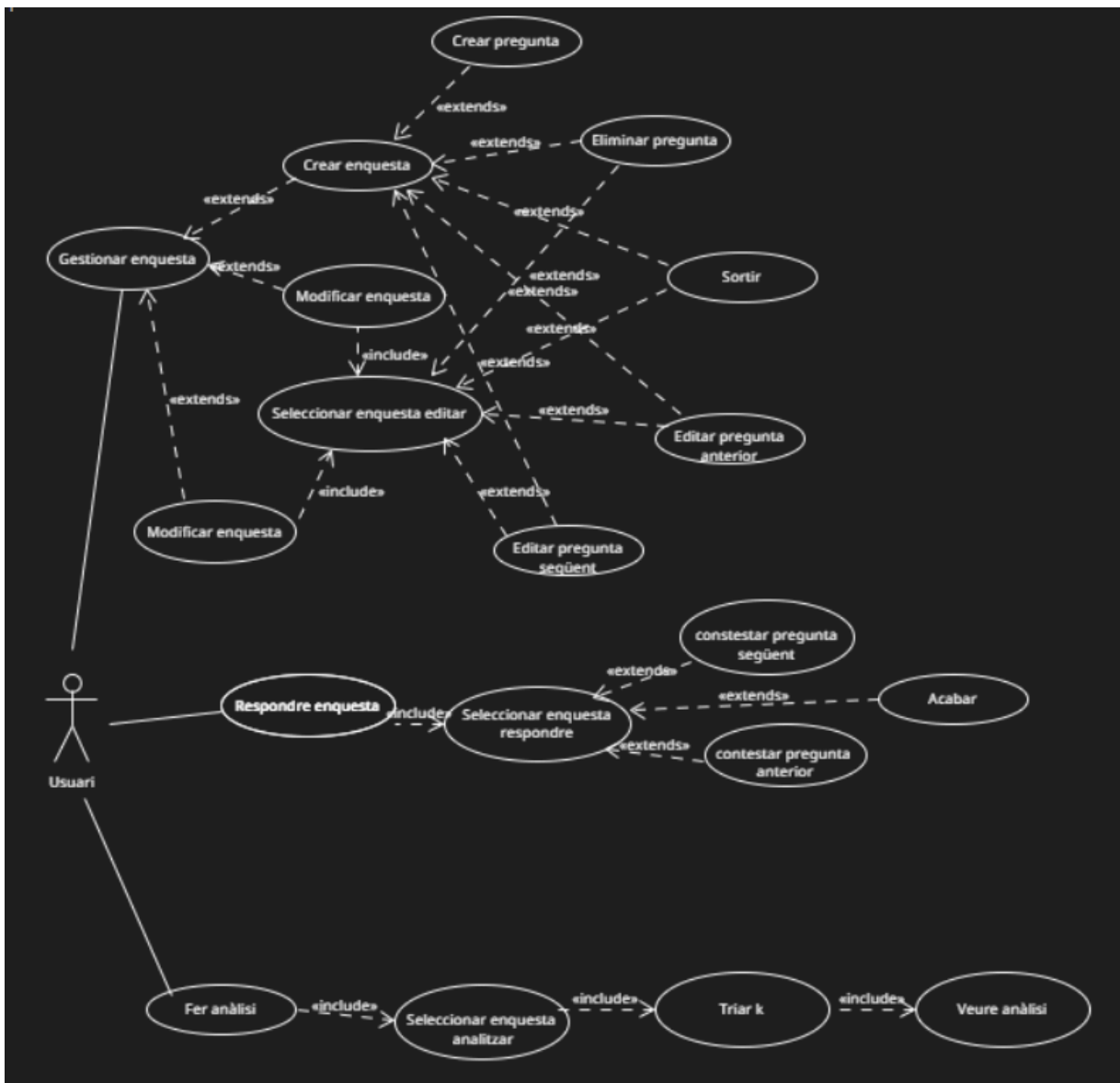
Nom: Acabar

- Actor: Usuari
- Comportament: Es guarda la nova resposta i es torna a la pàgina principal.
- Errors possibles i cursos alternatius: Si no s'han respost totes les preguntes aquestes es desen en blanc.

MANUAL D'USUARI

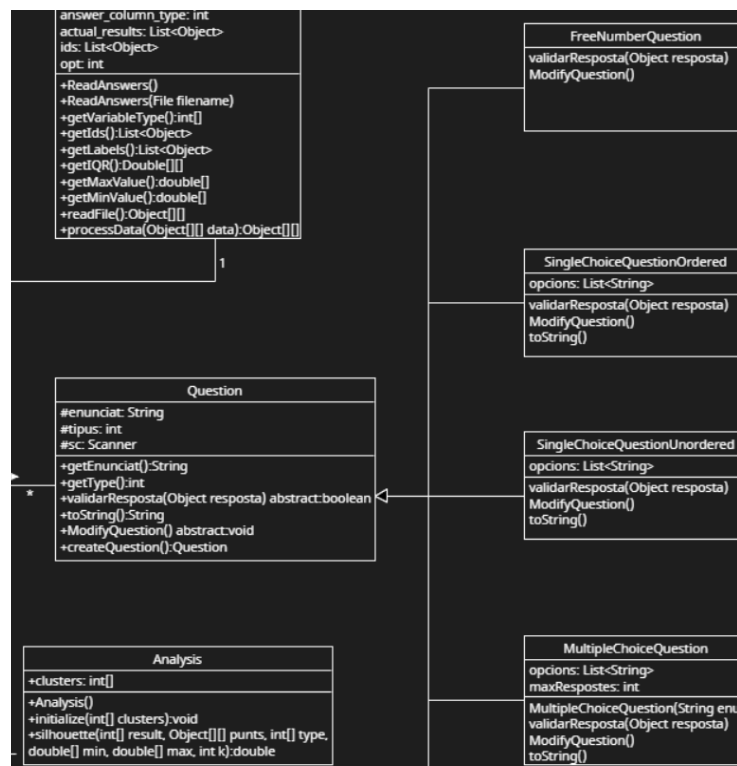
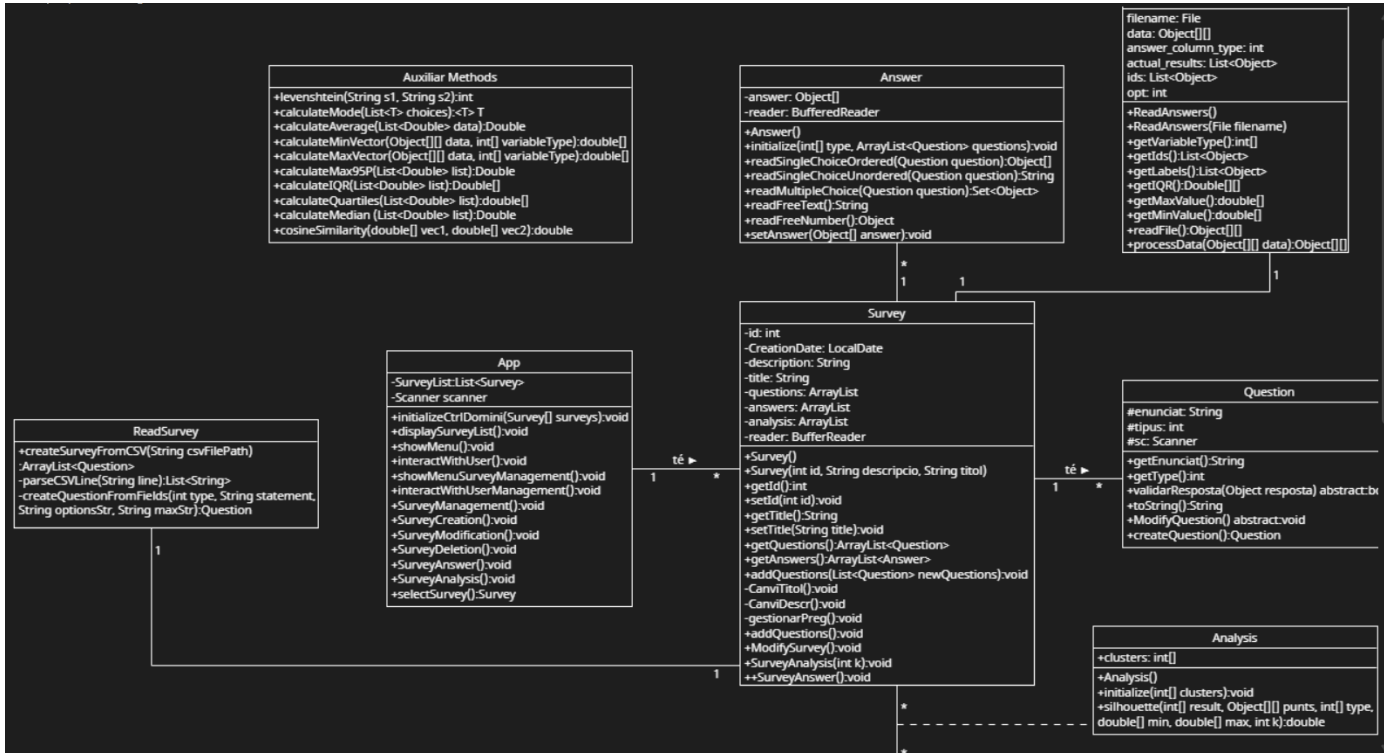
Hi ha un document a DOCS anomenat "Manual d'usuari" que conté una explicació extensiva sobre el funcionament de l'aplicació.

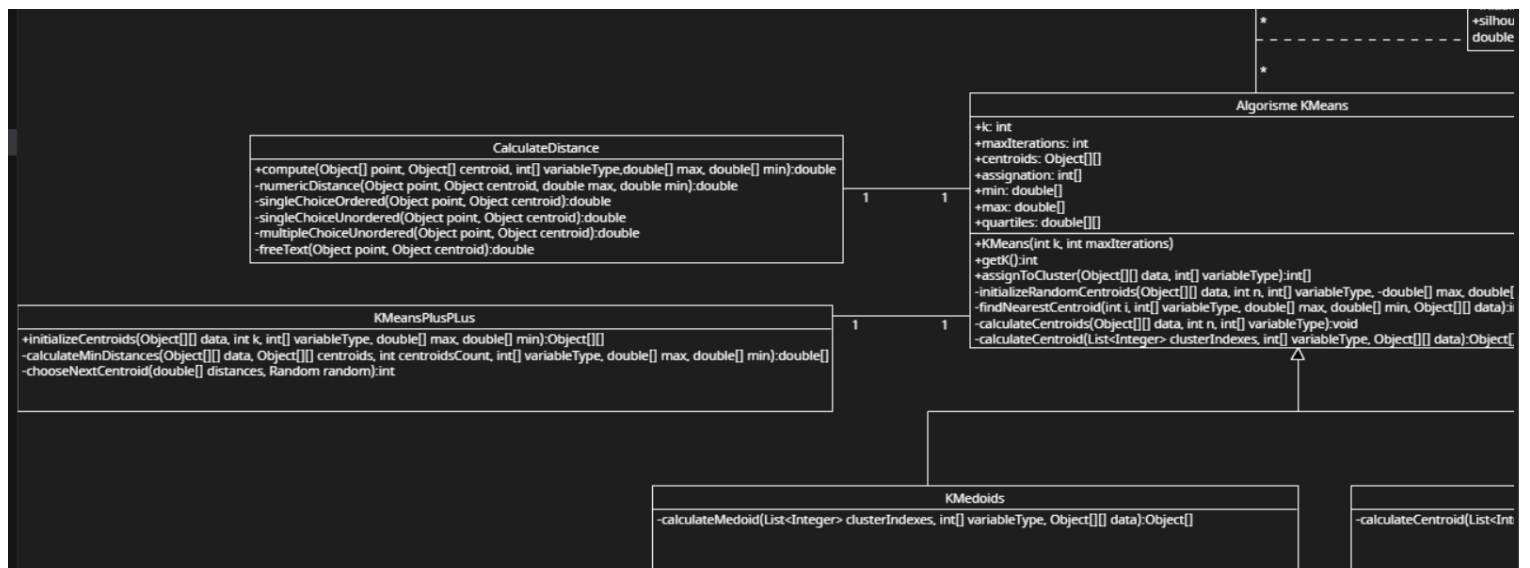
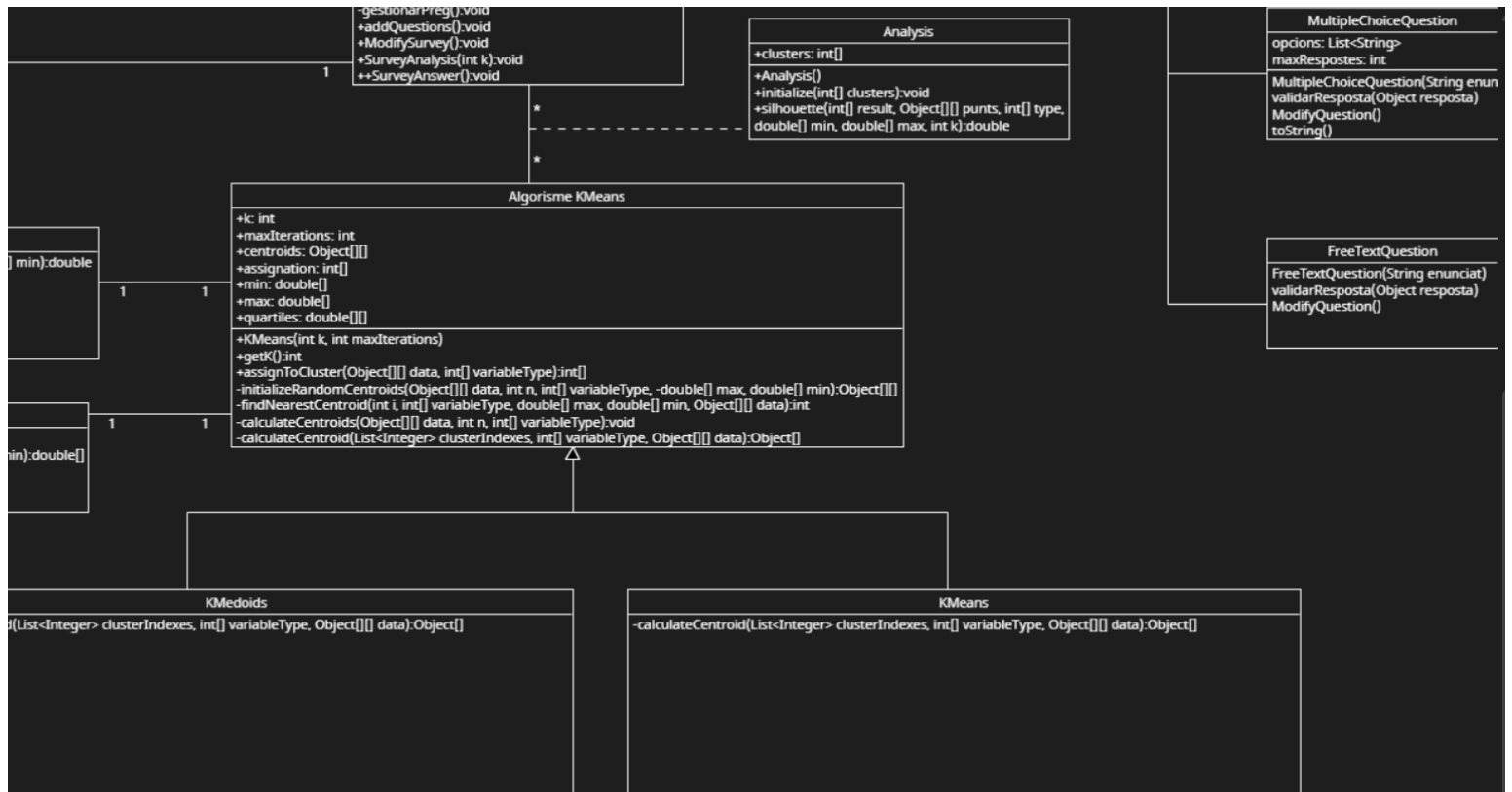
DIAGRAMA CASOS DÚS



Ja que el diagrama de classes és un fitxer molt gran, per poder veure-ho amb una bona qualitat caldrà ampliar la imatge. És per això que a continuació es pot observar la imatge completa i, a la pròxima pàgina, separada en parts per a permetre la llegibilitat.







CLASSES IMPLEMENTADES PER CADASCÚ

Paula → CalculateDistance, AuxiliarMethods, Algorithm, KMedoids, KMeans, KMeansPlusPlus, TestAccuracy, KMeansDriver, CalculateDistancesTest, AuxiliarMethodsTest

Abel → ReadAnswers, Question, Main, Makefile, QuestionTest, SurveyTest, PanelMenu., VistaCrearEnquesta, VistaCrearPregunta, VistaEditorPregunta, VistaEsborrarEnquesta, VistaGestionar, VistaModificarEnquesta, VistaRespondreEnquesta, VistaSeleccionarEnquesta, VistaPrincipal, CtrlPresentacio

Judit → Survey, Analysis, GestorDeDades, LocalDateAdapter, SurveyData.

Aura → Answer, CtrlDomini, ReadSurvey, AnswerTest, AnalysisTest.java, KMeansTest.java, KMedoidsTest.java, ReadAnswersTest.java, SurveyAnalysisTest.java, CtrlDominiTest.java, CtrlDominiHeadlessTest.java, LocalDateAdapterTest.java, GestorDeDadesTest.java